

Workshop 2026
Lille Groupe 1 - L'Erreurscope
Documentation Technique

Sommaire

1. Présentation Générale du Projet	3
1.1 Contexte et Ancrage RickLab™	3
1.2 Problématique, Concept et Objectifs	3
1.3 Parcours et Expérience Utilisateur	3
2. Architecture Matérielle et Électronique	5
2.1 Architecture Globale du Système (Arduino Yùn & Raspberry Pi 3)	5
2.2 Nomenclature et Spécifications des Composants	5
2.3 Schéma Électronique et Brochage	6
2.4 Conception Structurale et Compartimentage (4 Étages)	6
3. Architecture Logicielle & Algorithmique	10
3.1 Structure du Code Embarqué Arduino (RFID, Servo, LCD)	10
3.2 Module Traitement Vocal & Audio Python (Raspberry Pi)	12
3.3 Intégration de l'Intelligence Artificielle	12
3.4 Architecture de la Raspberry	12
4. Conception Mécanique & Fabrication Numérique	14
4.1 Découpe Laser et Modélisation 3D	14
4.2 Historique du Prototypage et Évolution	14
5. Gestion de Projet & Organisation	21
5.1 Répartition des Rôles et Rituels d'Équipe	21
5.2 Suivi des Phases de Développement	21
6. Conclusion et Perspectives	26
Dépôt Git & Ressources	26
7. Annexes	27
7.1 Logiciels & Outils Numériques	27
7.2 Technologies & Protocoles	27
7.3 Matériels, Outillage & Consommables	27

1. Présentation Générale du Projet

1.1 Contexte et Ancrage RickLab™

Le projet **L'Erreurscope** s'inscrit pleinement dans le cadre académique et créatif du Workshop 2026 B2. Il s'ancre directement dans la philosophie du **RickLab™** : concevoir un dispositif technologique à la fois parfaitement fonctionnel sur le plan de l'ingénierie, interactif et délibérément absurde ou cynique.

Inspiré par l'univers scientifique satirique et déjanté de la série *Rick & Morty*, L'Erreurscope combine de la mécatronique interactive, de la reconnaissance vocale et du traitement audio afin de proposer une expérience utilisateur mémorable.

1.2 Problématique, Concept et Objectifs

L'objectif principal de L'Erreurscope est de moduler de manière sarcastique l'état d'esprit de l'utilisateur au début de sa journée. Plutôt que de fournir des prédictions astrologiques bienveillantes ou réconfortantes, le dispositif délivre un "horoscope d'erreur" conçu pour rabaisser l'estime de soi de façon humoristique, quel que soit le signe ou l'identité de la personne.

Le concept repose sur un coffret en bois interactif fermé à clé mécanique et électronique. Pour accéder aux fonctions de prédiction, l'utilisateur doit résoudre une énigme tangible. Une fois la boîte déverrouillée, un système d'acquisition vocale guidé par un bouton interactif ("Appuyer-pour-parler") enregistre l'identité et la date de naissance de l'utilisateur, puis génère et restitue vocalement une sentence ironique ciblée.

1.3 Parcours et Expérience Utilisateur

L'expérience utilisateur se déroule selon la séquence suivante :

- **1. Énigme des Mouches¹ (Authentification RFID)** : L'utilisateur fait face à la boîte fermée. Sur le plateau supérieur, plusieurs mouches décoratives imprimées en 3D sont présentes. L'utilisateur doit manipuler et positionner la "bonne" mouche (contenant un tag RFID dissimulé) au-dessus de la zone du capteur RFID. Dès la détection validée, une LED verte s'allume, l'écran LCD affiche "Accès autorisé !" et un servomoteur déclenche un levier mécanique qui soulève le couvercle supérieur.

- **2. Acquisition Vocale ("Appuyer-pour-parler")** : Le couvercle ouvert révèle l'interface principale composée d'un écran LCD, de deux haut-parleurs intégrés et d'un bouton bleu interactif. L'utilisateur maintient le bouton enfoncé pour s'exprimer dans le microphone en énonçant son prénom et sa date de naissance.
- **3. Traitement et Retranscription** : Le signal audio est capturé et analysé par un script Python hébergé sur le Raspberry Pi 3. L'écran retranscrit le texte identifié et détermine le signe astrologique correspondant
- **4. Restitution Sonore** : Le système sélectionne et joue une réplique audio acide et sarcastique via les deux haut-parleurs de la boîte, finalisant l'expérience.

Mouches¹ : la référence vient de Rick, qui lors d'un épisode, mélange et assemble une combinaison de mouches, ce qui fait apparaître un lab magique dans son garage.

2. Architecture Matérielle et Électronique

2.1 Architecture Globale du Système (Arduino Yùn & Raspberry Pi 3)

Afin d'assurer à la fois une réactivité temps réel pour le pilotage des capteurs/actionneurs et une puissance de calcul suffisante pour le traitement audio, le système repose sur une architecture hybride coopérative :

- **Microcontrôleur Arduino Yùn** : Prend en charge l'acquisition temps réel du lecteur RFID RC522, le contrôle du servomoteur de verrouillage, la gestion des indicateurs lumineux (LED) et l'affichage des messages d'état sur l'écran LCD 16x2.
- **Raspberry Pi 3** : Assure le rôle d'unité centrale pour la capture du signal microphone, l'exécution du script d'analyse vocale Python, le traitement algorithmique et l'amplification audio vers les deux haut-parleurs.

2.2 Nomenclature et Spécifications des Composants

- **Arduino Yùn** : Unité de contrôle matériel principale de l'ouverture de la boîte.
- **Raspberry Pi 3** : Unité de traitement audio, speech-to-text et logique d'horoscope avec le script en Python.
- **Module RFID RC522** : Lecteur de cartes/tags communiquant via bus SPI.
- **Servomoteur** : Actionneur de levage mécanique du couvercle piloté.
- **Afficheur LCD 16x2** : Écran alphanumérique parallèle pour l'affichage d'instructions.
- **Bouton Poussoir & Microphone** : Interface d'enregistrement vocal de type "Appuyer-pour-parler".
- **Duo de Haut-Parleurs** : Restitution sonore amplifiée des prédictions audio.
- **LED Verte** : Témoin d'accès et de détection RFID.

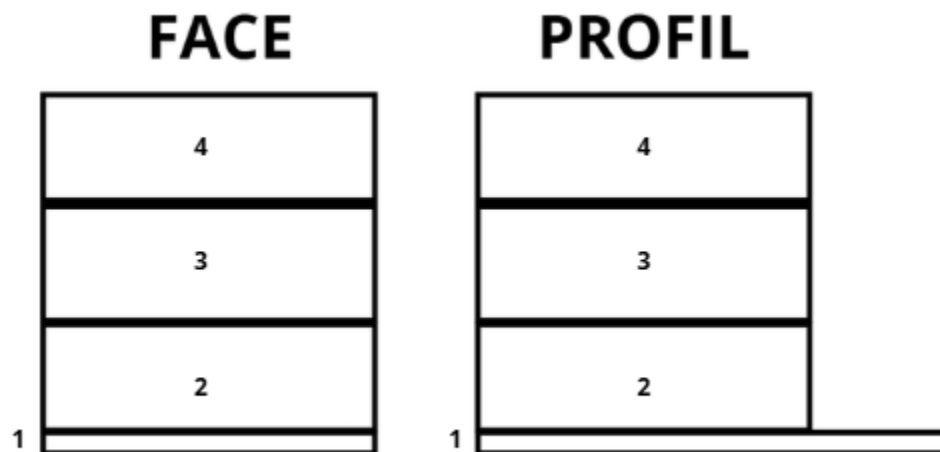
- **Breadboard & Câblage Dupont** : Interconnexions électriques et rails de distribution 5V/3.3V/GND.

2.3 Schéma Électronique et Brochage

Le circuit intègre une alimentation distribuée (5V pour le servomoteur, l'écran LCD et le Raspberry Pi ; 3.3V pour le module RFID). Le module RFID MFRC522 est relié à l'Arduino via le bus SPI (broches 9 à 13). L'écran LCD 16x2 est piloté en mode 4 bits (broches 3, 4, 5, 6, 7, 8). Le servomoteur est raccordé à la broche A0 (utilisée en sortie numérique PWM) et la LED de validation sur la broche digitale 2.

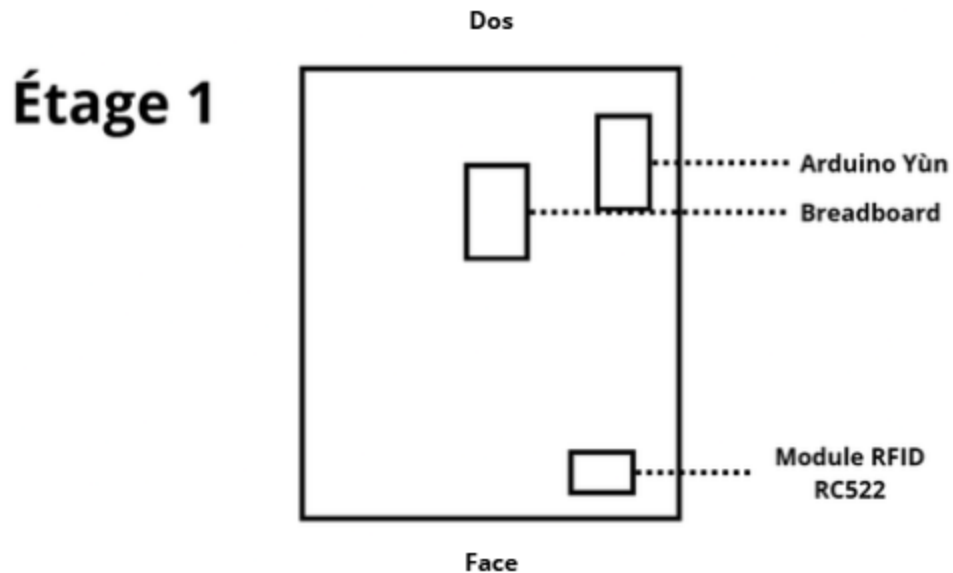
2.4 Conception Structurale et Compartimentage (4 Étages)

Pour garantir une intégration propre du câblage, une répartition optimale de la masse et un accès d'entretien aisé, la structure interne du coffret en bois a été modélisée et découpée en 4 étages fonctionnels :



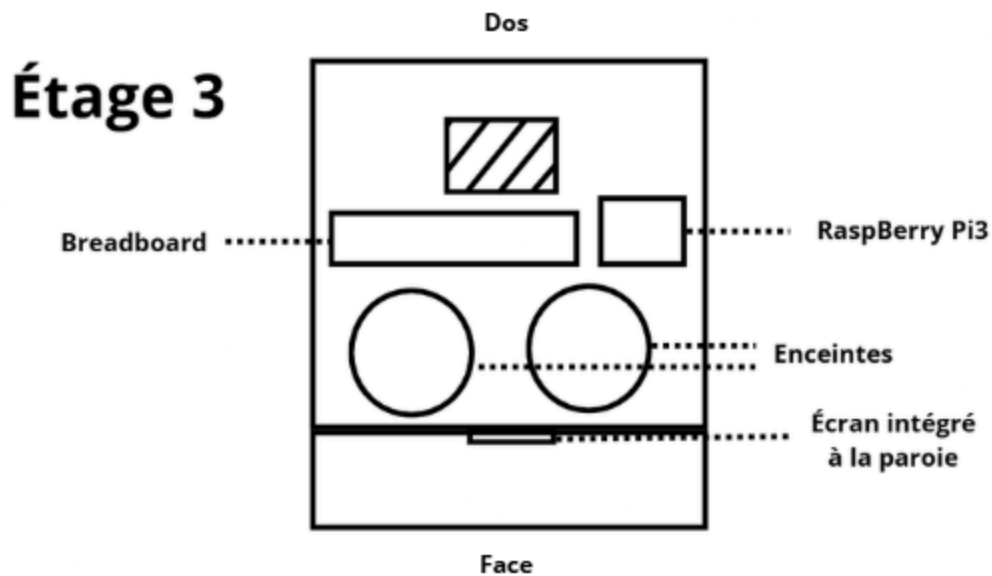
Vue d'ensemble de la structure : Face et Profil

- **Étage 1 (Base inférieure)** : Logement de l'électronique de puissance et de contrôle matériel, contenant la carte Arduino Uno, le breadboard principal de distribution électrique et le module RFID RC522 orienté vers la plaque supérieure.



Disposition des composants à l'Étage 1

- **Étage 2 (Plateau intermédiaire)** : Étape "invisible" servant de surface de pose pour les mouches décoratives et assurant l'isolation physique entre la base électronique et les étages supérieurs.
- **Étage 3 (Unité centrale et Audio)** : Contient le Raspberry Pi 3, une breadboard secondaire, les deux enceintes haut-parleurs orientées vers l'avant et l'arrière de l'écran intégré à la paroi.



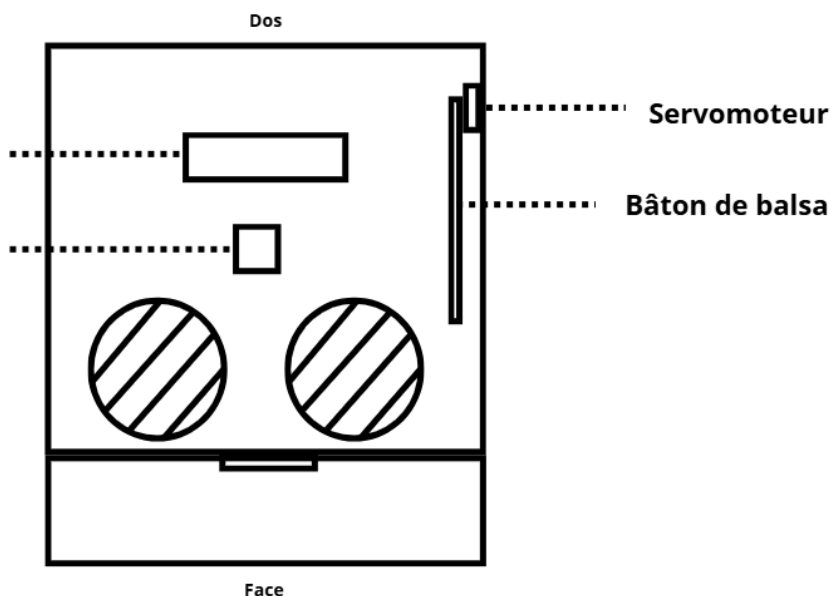
Disposition des composants à l'Étage 3

- **Étage 4 (Interface utilisateur supérieure)** : Comprend l'écran LCD 16x2 encastré dans son support 3D, le bouton poussoir bleu, les ouvertures de haut-parleurs et le servomoteur couplé au bâton de balsa servant de levier de déverrouillage du couvercle.

Étage 4

Écran dans son
support 3D

Bouton



Disposition de l'interface supérieure à l'Étage 4

3. Architecture Logicielle & Algorithmique

3.1 Structure du Code Embarqué Arduino (RFID, Servo, LCD)

Le programme Arduino embarqué gère la séquence d'accès sécurisé et le contrôle mécanique :

- **Inclusions & Configurations** : Intégration des bibliothèques matérielles **SPI.h**, **MFRC522.h**, **LiquidCrystal.h** et **Servo.h**.

```

1  #include <SPI.h>
2  #include <MFRC522.h>
3  #include <LiquidCrystal.h>
4  #include <Servo.h>
5
6  // --- Définition des broches ---
7  #define RST_PIN 9
8  #define SS_PIN 10
9  #define LED_PIN 2
10 #define SERVO_PIN A0
  
```

Figure 14 : Inclusions de bibliothèques et définitions des broches Arduino

- **Routine setup()** : Initialisation des bus de communication, configuration des E/S, réinitialisation du servo à 0° (position fermée) puis détachement du signal PWM (detach) pour éviter la chauffe et les vibrations parasites.

```

12 // --- Initialisation des composants ---
13 MFRC522 rfid(SS_PIN, RST_PIN); // Création de l'objet RFID
14 LiquidCrystal lcd(8, 7, 6, 5, 4, 3); // Création de l'objet LCD avec ses broches (RS, EN, D4, D5, D6, D7)
15 Servo monServo; // Création de l'objet servomoteur
16
17 void setup() {
18     SPI.begin(); // Initialisation du bus SPI
19     rfid.PCD_Init(); // Initialisation du lecteur RFID
20
21     pinMode(LED_PIN, OUTPUT); // Configuration de la broche LED en sortie
22
23     // Initialisation du servomoteur (fermé par défaut)
24     monServo.attach(SERVO_PIN);
25     monServo.write(0); // Met le servo à 0°
26     delay(300); // Attente pour laisser le moteur se positionner
27     monServo.detach(); // Coupe le signal pour éviter qu'il ne tremble
28
29     // Initialisation de l'écran LCD
30     lcd.begin(16, 2); // Définit l'écran comme un modèle 16 colonnes x 2 lignes
31     affichageParDefaut();
32 }
  
```

Figure 15 : Instanciation des objets et routine d'initialisation setup()

- **Boucle principale loop()** : Scrutation active du lecteur RFID. Lors de la présentation de la mouche valide, activation de la LED verte, affichage de "Accès autorisé !" sur le LCD et rotation du servo à 140° pour soulever la trappe. Un algorithme avec tolérance de 3 échecs consécutifs maintient l'ouverture tant que le badge est présent.

```

34 void loop() {
35     // Vérifie si un NOUVEAU badge est physiquement présent ET si on peut lire son ID
36     if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
37
38         // --- 1. OUVERTURE ---
39         lcd.clear();
40         lcd.print("Acces autorise !");
41         digitalWrite(LED_PIN, HIGH); // Allume la LED
42
43         monServo.attach(SERVO_PIN); // Réactive le servomoteur
44         monServo.write(140);        // Ouvre à 140°
45
46         rfid.PICC_HaltA(); // Met le badge en pause pour ne pas le lire en boucle infinie
47
48         delay(300);          // Laisse le temps au servo de s'ouvrir
49         monServo.detach();    // Coupe le signal pour éviter les vibrations et conflits SPI
50
51         // --- 2. MAINTIEN (Tant que le badge est là) ---
52         int echecsLecture = 0; // Compteur pour vérifier si le badge a disparu
53
54         while (echecsLecture < 3) { // Tolérance : 3 échecs consécutifs = badge retiré
55             delay(300);
56
57             byte bufferATQA[2];
58             byte bufferSize = sizeof(bufferATQA);
59
60             // PICC_WakeupA envoie un "ping" au badge endormi.
61             // S'il répond (STATUS_OK), c'est qu'il est toujours sur le lecteur.
62             if (rfid.PICC_WakeupA(bufferATQA, &bufferSize) == MFRC522::STATUS_OK) {
63                 rfid.PICC_HaltA(); // Le badge est bien là, on le rendort
64                 echecsLecture = 0; // On réinitialise le compteur d'erreurs
65             } else {
66                 echecsLecture++; // Le badge ne répond pas, on incrémente l'erreur
67             }
68         }
69     }

```

Figure 16 : Algorithme de détection et maintien d'ouverture dans la boucle loop()

- **Fermeture & Réinitialisation** : Lors du retrait du badge, le servo retourne à 0°, la LED s'éteint et la fonction **affichageParDefaut()** réinitialise l'écran avec le message "Veuillez entrer code secret".

```

70 // --- 3. FERMETURE (Le badge a été retiré) ---
71 monServo.attach(SERVO_PIN); // Réactive le servomoteur
72 digitalWrite(LED_PIN, LOW); // Éteint la LED
73 monServo.write(0); // Referme à 0°
74 delay(300); // Laisse le temps de se fermer
75 monServo.detach(); // Coupe le signal
76
77 rfid.PCD_Init(); // Réinitialise complètement le lecteur RFID pour le prochain badge
78
79 affichageParDefaut(); // Remet le texte de base sur l'écran
80 }
81 }
82
83 // Fonction personnalisée pour simplifier l'affichage de base
84 void affichageParDefaut() {
85     lcd.clear();
86     lcd.print("Veuillez entrer");
87     lcd.setCursor(0, 1); // Passe à la 2ème ligne (colonne 0, ligne 1)
88     lcd.print("code secret");
89 }

```

Figure 17 : Routine de fermeture sécurisée du servo et fonction d'affichage par défaut

3.2 Module Traitement Vocal & Audio Python (Raspberry Pi)

Le Raspberry Pi 3 orchestre l'interaction audio via un script Python dédié. Lorsqu'une pression sur le bouton bleu est détectée, le flux audio provenant du microphone est enregistré. Le fichier audio est ensuite analysé par un moteur de reconnaissance vocale ("Speech-to-Text") afin de déduire la date de naissance et le signe astrologique associé. Une fois le signe identifié, le script sélectionne une réplique audio acide préenregistrée et la restitue via les haut-parleurs.

3.3 Intégration de l'Intelligence Artificielle

Au cours du développement, les modèles d'intelligence artificielle (Claude et Gemini) ont été exploités pour structurer la logique algorithmique, optimiser le code Arduino et concevoir le script d'interaction Python. De plus, l'IA a permis de générer la trame rédactionnelle des répliques sarcastiques de l'horoscope.

3.4 Architecture de la Raspberry

Voici l'architecture de la structure du code de la raspberry :


```
rick@raspberrypi:/home/pi $ tree
.
├── erreurscope
│   ├── audios
│   │   ├── 1.mp3
│   │   ├── 2.mp3
│   │   ├── 3.mp3
│   │   └── 4.mp3
│   ├── testHP2.py
│   └── testHP.py
├── test
│   ├── test10.py
│   ├── testHP.py
│   ├── test.py
│   └── vosk-model-small-fr-0.22
│       ├── am
│       │   └── final.mdl
│       ├── conf
│       │   ├── mfcc.conf
│       │   └── model.conf
│       ├── graph
│       │   ├── disambig_tid.int
│       │   ├── Gr.fst
│       │   ├── HCLr.fst
│       │   ├── phones
│       │   │   └── word_boundary.int
│       ├── ivector
│       │   ├── final.dubm
│       │   ├── final.ie
│       │   ├── final.mat
│       │   ├── global_cmvn.stats
│       │   ├── online_cmvn.conf
│       │   └── splice.conf
│       └── README
└── vosk-model-small-fr-0.22.zip

10 directories, 24 files
rick@raspberrypi:/home/pi $ |
```

4. Conception Mécanique & Fabrication Numérique

4.1 Découpe Laser et Modélisation 3D

Le prototypage et la fabrication matérielle de L'Erreurscope ont été entièrement réalisés au sein du **myDiL** :

- **Découpe Laser** : Usinage de précision de panneaux en bois de contreplaqué (balsa) pour constituer le coffret extérieur, les cloisons des 4 étages et la gravure thématique du disque astrologique sur le couvercle.
- **Impression 3D** : Conception et impression des éléments clés : mouches décoratives (intégrant le badge RFID), cadre d'encastrement LCD, bouton poussoir et charnières mécaniques de structure.

4.2 Historique du Prototypage et Évolution

Le projet a suivi une démarche de prototypage agile et itératif :

- **Étape 1 : Validation Électronique sur Breadboard** : Câblage initial et validation des communications.

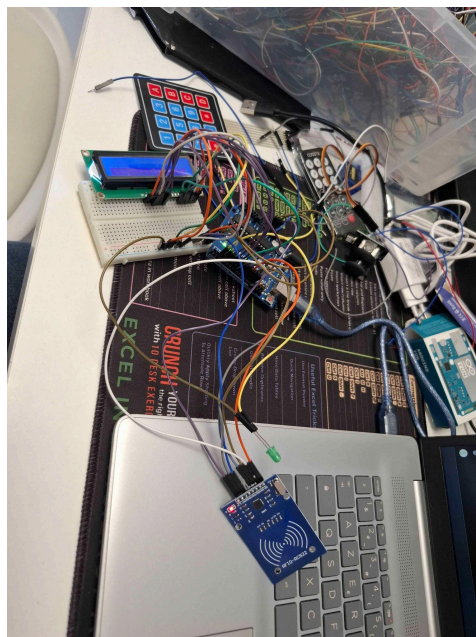


Figure 1 : Assemblage initial des composants électroniques sur breadboard

- **Étape 2 : Prototype Fonctionnel en Carton** : Maquette volumétrique pour tester l'ergonomie, l'encastrement LCD et le système de déverrouillage mécanique.



Figure 2 : Prototype en carton avec écran LCD (gauche) et mécanisme d'ouverture à servomoteur (droite)

- **Étape 3 : Fabrication et Assemblage** : Découpe des planches et ajustement des panneaux de contreplaqué.

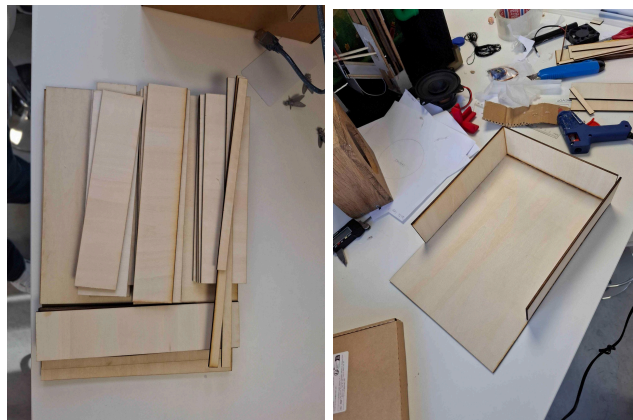


Figure 3 : Panneaux en bois découpés au laser et assemblage de la structure de base

- **Étape 4 : Finition Structurale et Intégration** : Assemblage des compartiments et découpes des ouvertures haut-parleurs.

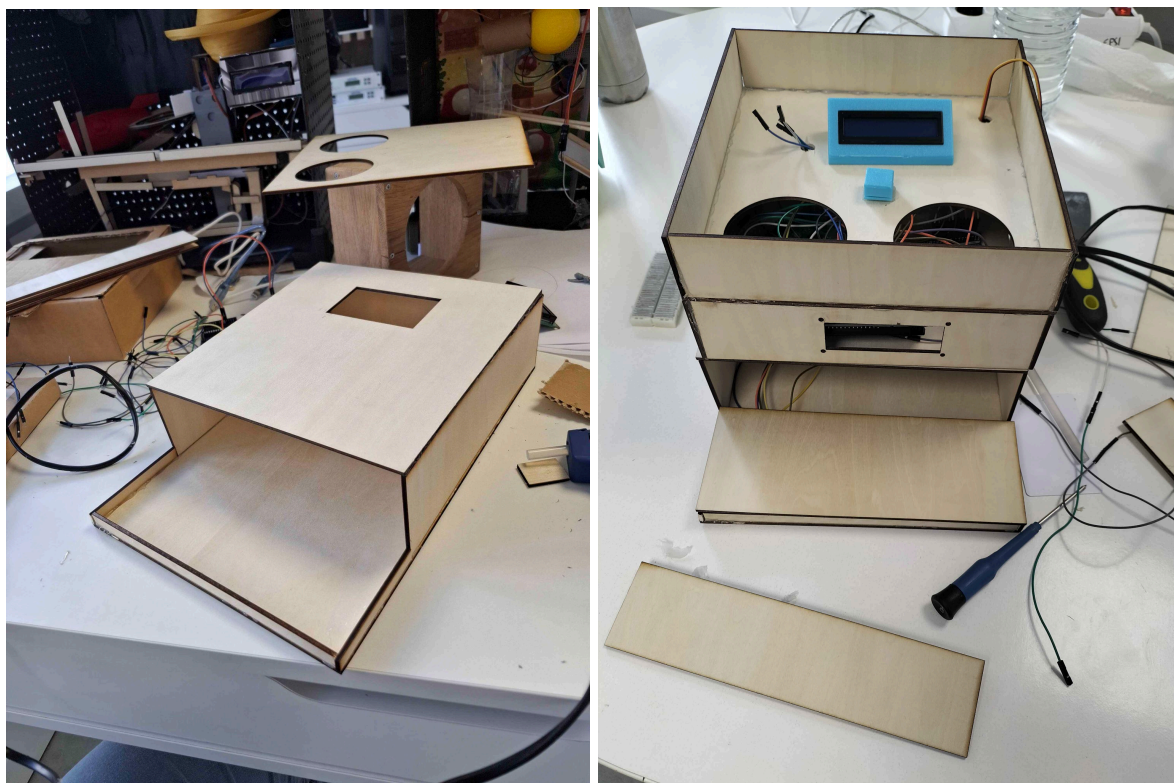
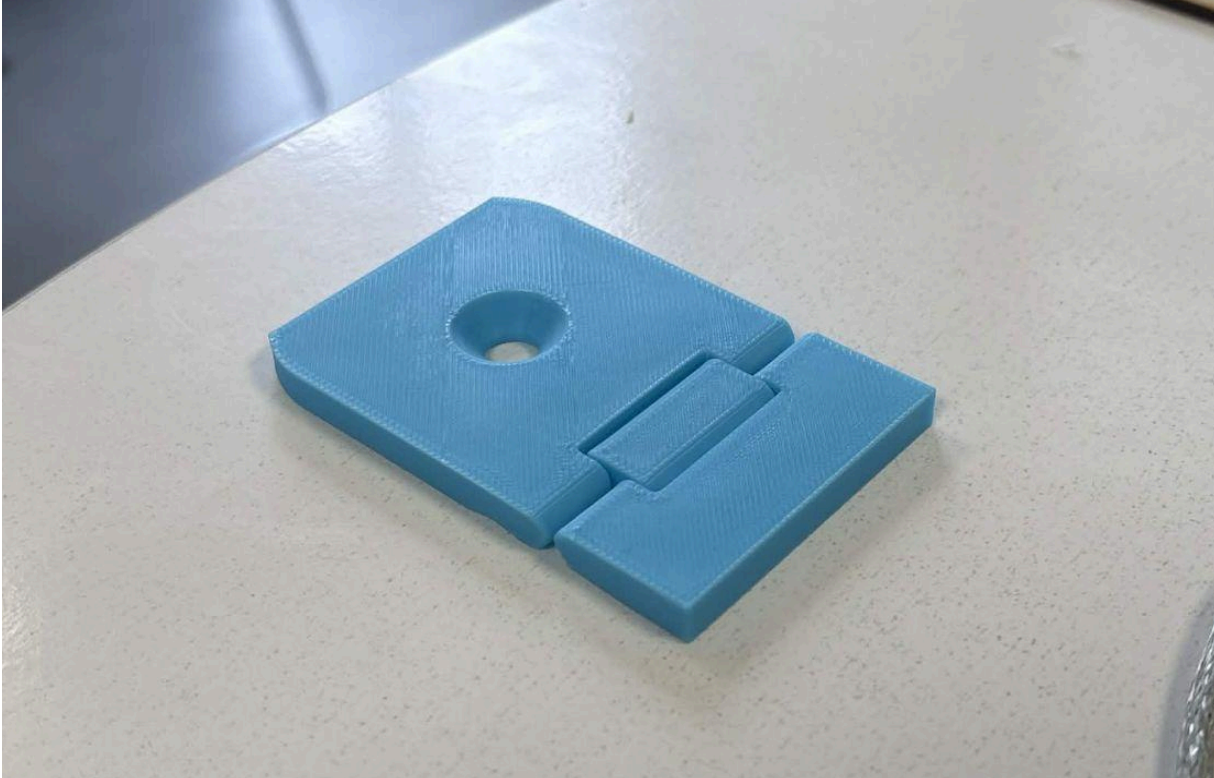


Figure 4 : Vues de la structure bois à double étage et compartiments internes

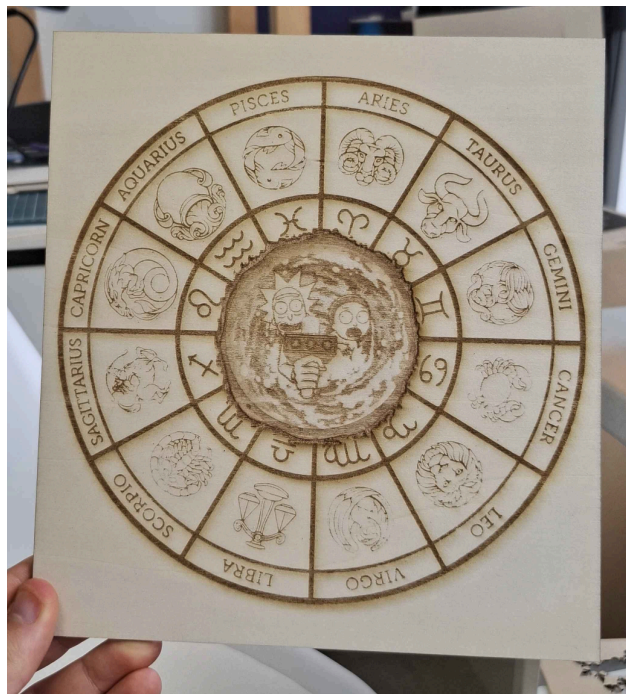
- **Étape 5 : Conception des Éléments d'Articulation 3D** : Évolution de la conception des charnières pour garantir solidité et intégration esthétique.





Charnières 3D, supports LCD et mouches imprimées

- **Étape 6 : Gravure Laser du Couvercle** : Gravure détaillée du cadran horoscope RickLab™.



Couvercle en bois découpé et gravé au laser

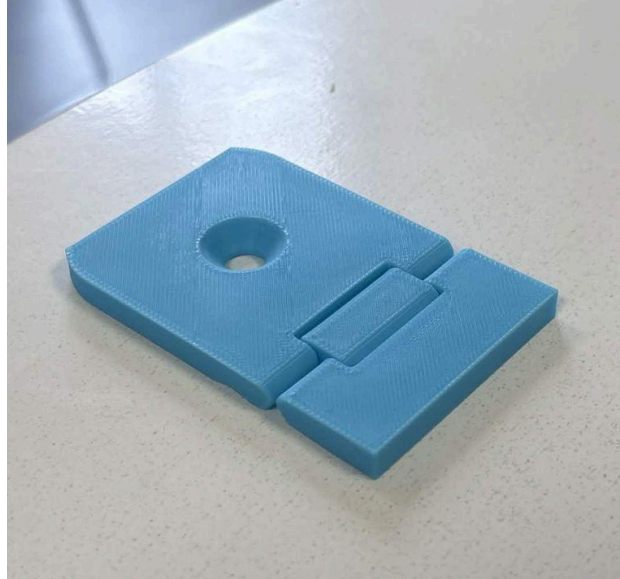
- **Résolution des Défis Mécaniques** : L'intégration des charnières a nécessité plusieurs révisions. Le premier modèle étant trop fragile :



- Une seconde version plus robuste a été développée :



- Enfin, pour s'adapter aux contraintes d'espace intérieur, un modèle spécifique encastré a été retenu pour le couvercle.



5. Gestion de Projet & Organisation

5.1 Répartition des Rôles et Rituels d'Équipe

Le projet s'est appuyé sur la méthodologie Agile avec la tenue quotidienne de *daily stand-up meetings* (10 minutes chaque matin) pour faire le bilan de la veille et ajuster les objectifs. La répartition des compétences au sein de l'équipe s'est structurée ainsi :

- **Louis** : Idéation du concept d'horoscope, gestion documentaire, conception graphique de l'affiche et gravures laser.
- **Cyril** : Modélisation et impression 3D des charnières et leviers, support de présentation et rédaction de la documentation technique.
- **Gautier** : Soudure des composants, câblage matériel, intégration du bouton et assistance à la préparation des présentations.
- **Théo** : Configuration du Raspberry Pi 3, scripts audio, traitement du signal vocal et calibrage du système sonore.
- **Yacine** : Conception des schémas électroniques, modélisation 3D du coffret, cotations laser, usinage, assemblage matériel et gestion du dépôt GitHub.

5.2 Suivi des Phases de Développement

Le déroulement du workshop s'est articulé sur quatre journées intensives documentées et centralisées par un repo Github, ainsi qu'au tableau blanc :

- **Lundi (Brainstorming & Prototypage rapide)** : Sélection de l'idée, câblage sur table, prototype carton et configuration initiale.

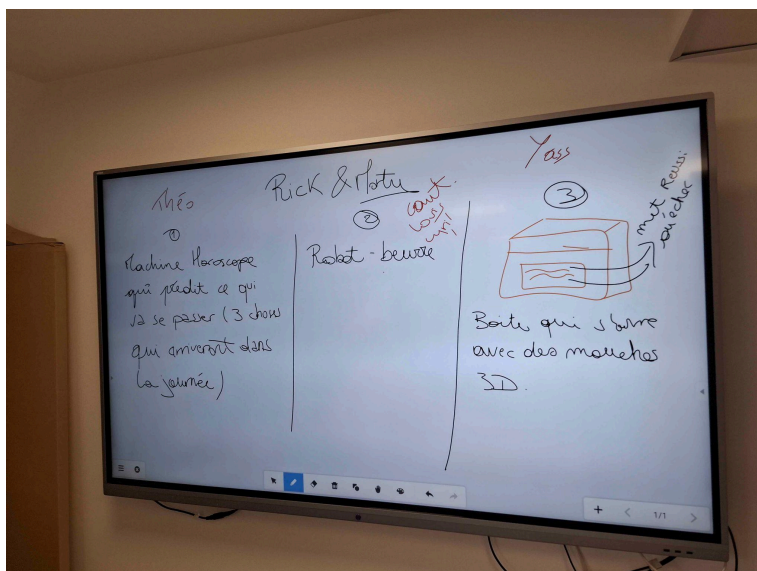


Figure 5 : Synthèse du brainstorming initial du Jour 1

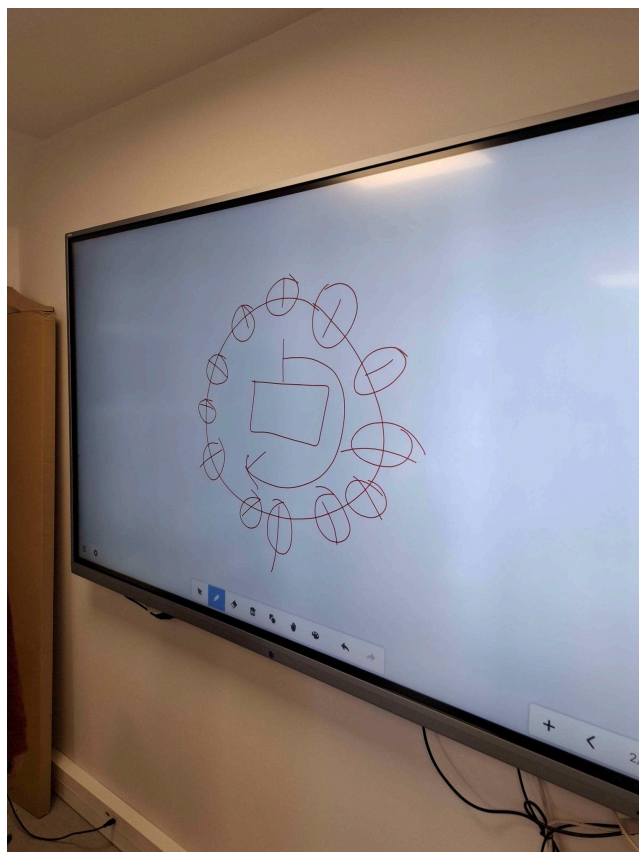


Figure 6 : Concept visuel de l'horoscope gravé sur le couvercle supérieur

- **Mardi (Modélisation & Conception Électronique) :** Schémas électroniques, cotations laser et scripts vocaux.

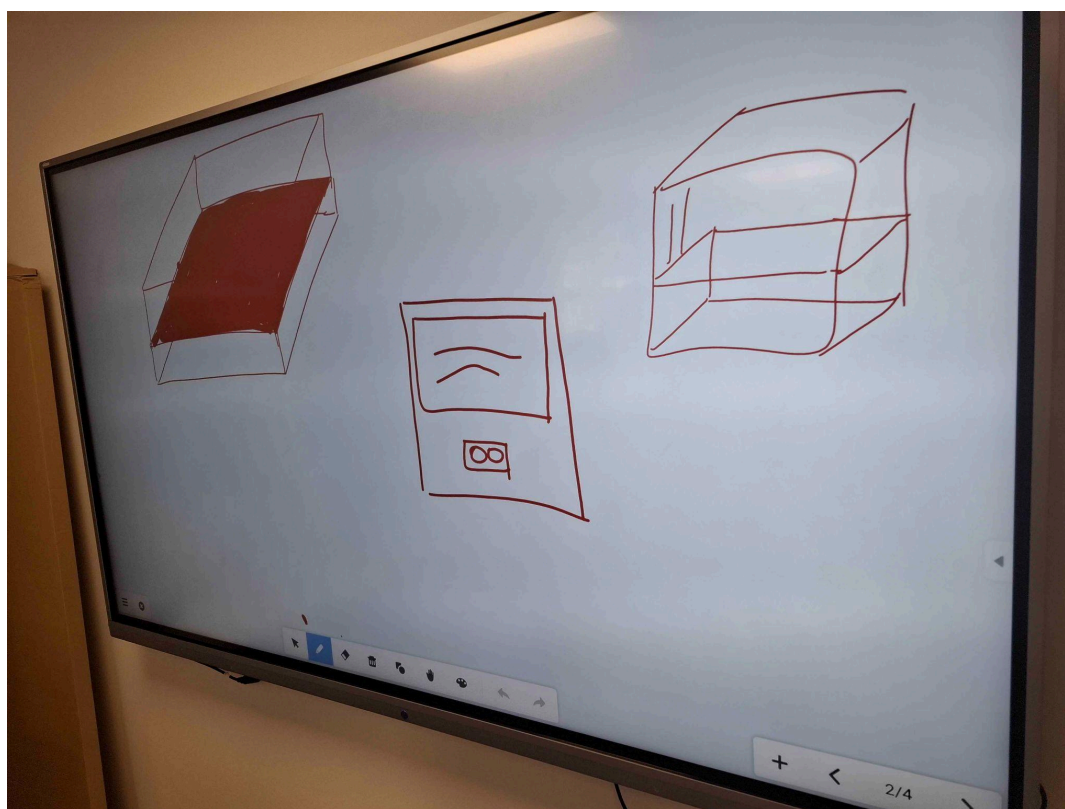
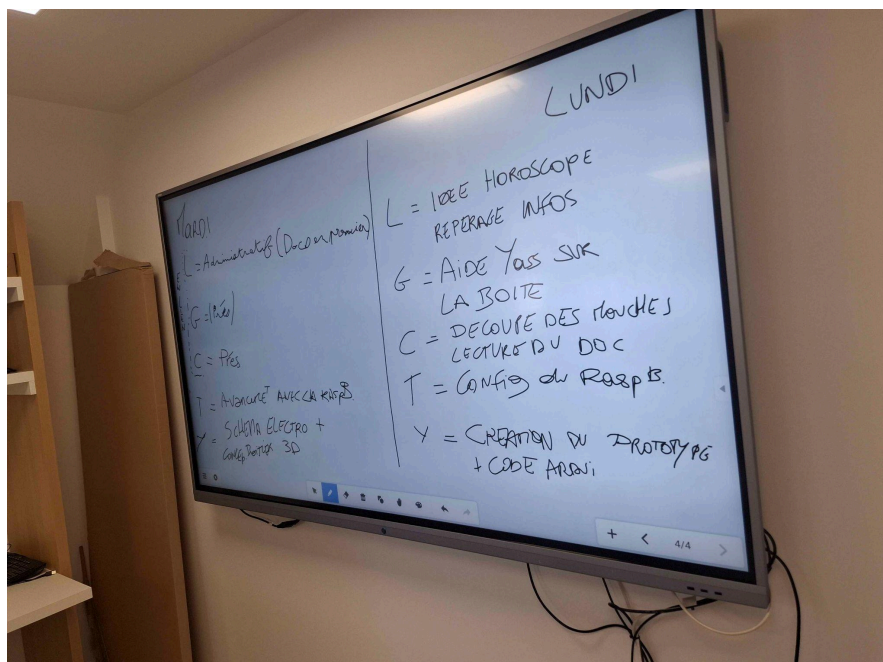


Figure 7 : Planning du lundi/mardi (gauche) et étude de la disposition par étages (droite)

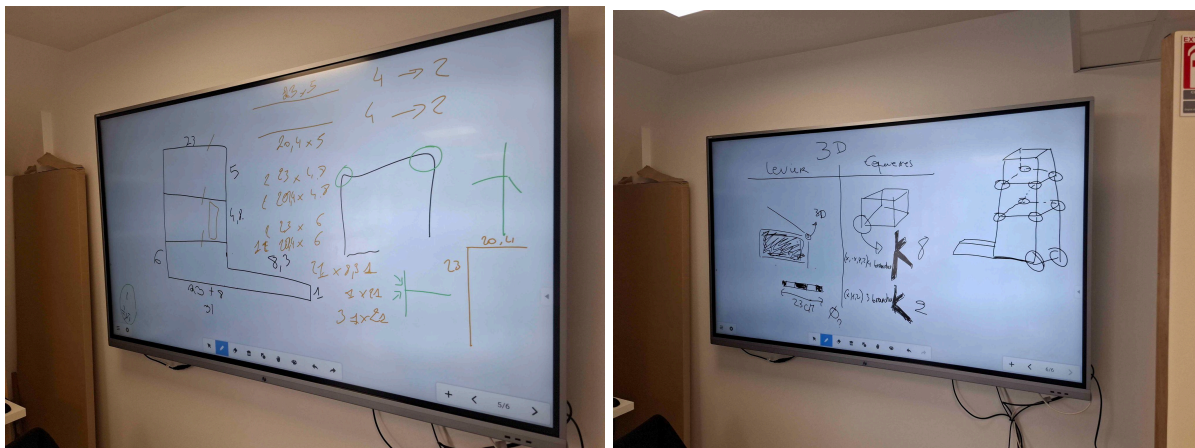


Figure 8 : Cotations pour la découpe laser (gauche) et modélisation des équerres & leviers (droite)

- **Mercredi (Fabrication & Assemblage) :** Découpe laser, impression 3D des pièces mécaniques et assemblage du coffret.

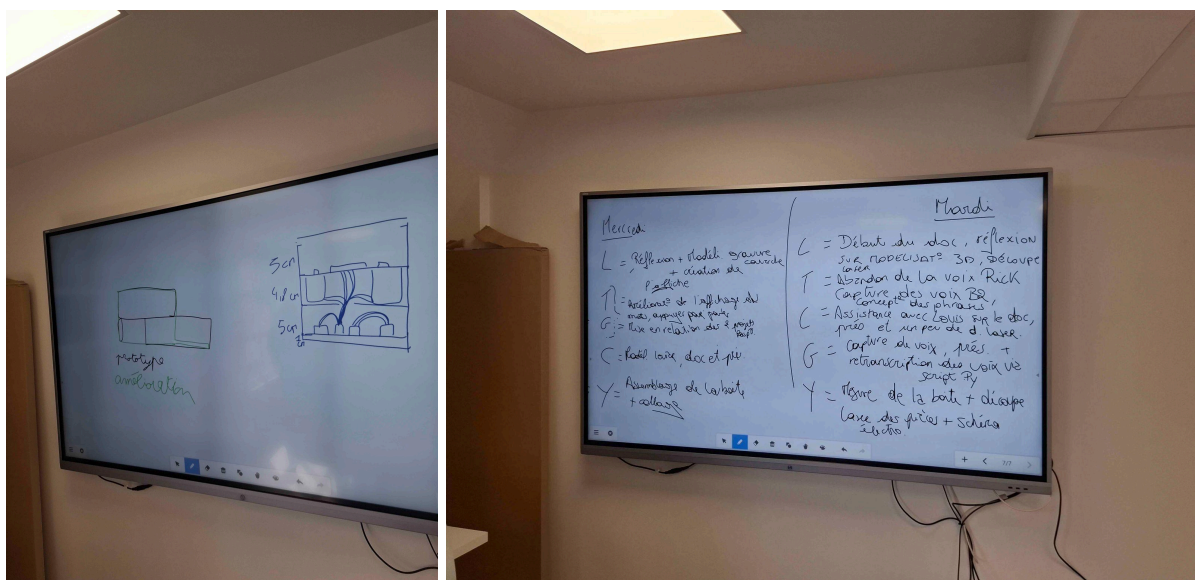


Figure 9 : Étude de compartimentage (gauche) et suivi des livrables du mercredi (droite)

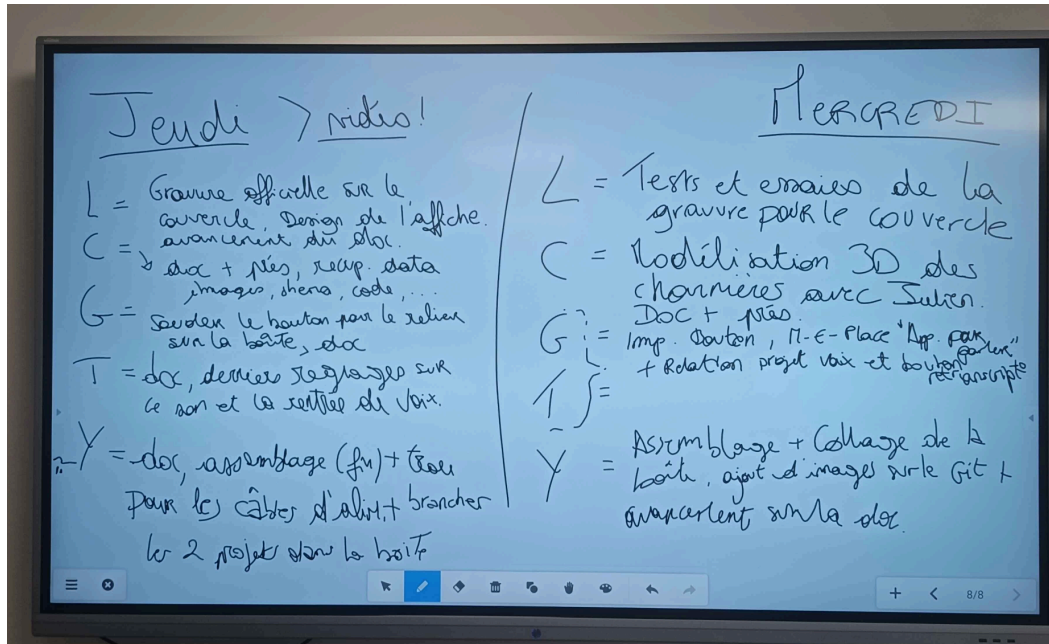
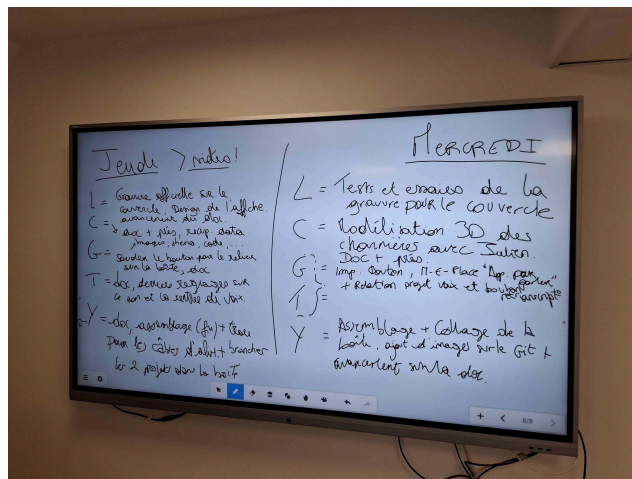
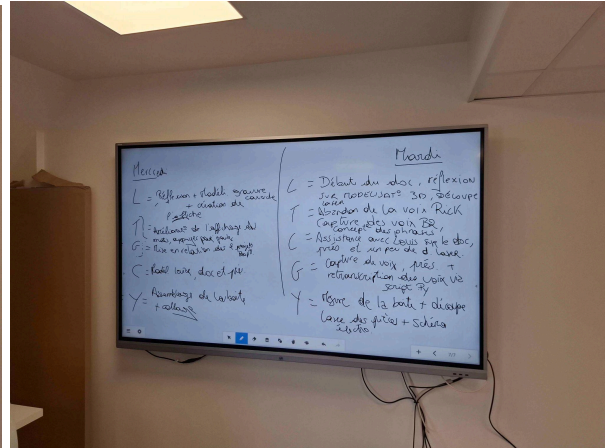
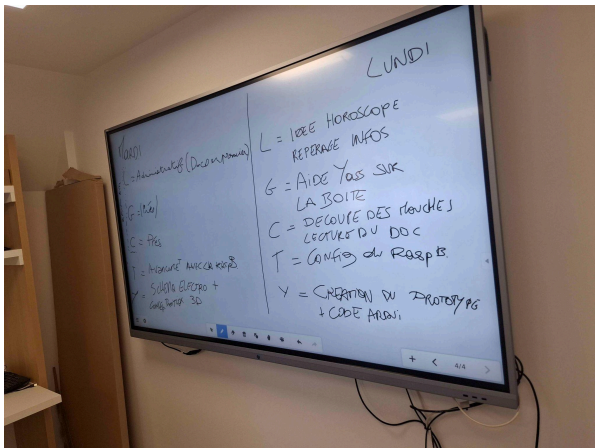


Figure 10 : Planning du mercredi/jeudi



Photos de suivi des rituels agiles sur tableau blanc

- **Jeudi (Finalisation & Imprévus Techniques)** : Gravure finale, ajustements audio et tests généraux. L'équipe a fait face à une panne critique de la carte SD du Raspberry Pi en fin d'après-midi, imposant la reconfiguration rapide d'une nouvelle carte et la réinstallation des scripts Python pour maintenir le projet opérationnel.

6. Conclusion et Perspectives

L'Erreurscope constitue un prototype abouti démontrant l'intégration réussie de systèmes embarqués Arduino, de traitement audio sur Raspberry Pi et de fabrication numérique. Plusieurs axes d'amélioration sont envisageables pour de futures versions :

- **Intégration interne du microphone** : Incorporer le microphone directement dans la structure de la boîte afin d'éviter les câbles externes et d'améliorer le rendu esthétique.
- **Finition et Peinture Thématique** : Appliquer une peinture et des visuels directement inspirés de l'univers *Rick & Morty* sur la structure en bois.
- **Génération Dynamique par IA** : Connecter le système à une API LLM en ligne pour générer des prédictions d'horoscope uniques et renouvelées à chaque utilisation.

Dépôt Git & Ressources

L'ensemble du code source, des scripts Python, des schémas et des modèles 3D est disponible publiquement :

Dépôt Officiel GitHub : <https://github.com/yacinehrc/L-erreurscope>

7. Annexes

7.1 Logiciels & Outils Numériques

- **Environnements de Développement (IDE) & CLI** : Arduino IDE, PowerShell
- **Intelligence Artificielle & Productivité** : Gemini, Claude, Suite Google (Drive, Docs)
- **Présentation & Design** : Canva, Prezi
- **Modélisation & Simulation (3D / CAD / Électronique)** : Fusion 360, Tinkercad, Wokwi
- **Plateformes 3D & Ressources en ligne** : MakerWorld, Thingiverse, YouTube

7.2 Technologies & Protocoles

- **Gestion de Version & Dépôt** : Git, GitHub
- **Réseau & Communication** : SSH

7.3 Matériels, Outillage & Consommables

- **Matériaux & Éco-conception** : Planches de Balsa (et chutes pour l'éco-conception), Boîtes en carton, PLA (impression 3D)
- **Outillage & Assemblage** : Pistolet à colle chaude, Colle

Fournitures de Mesure & Conception : Règle, Papier, Crayon